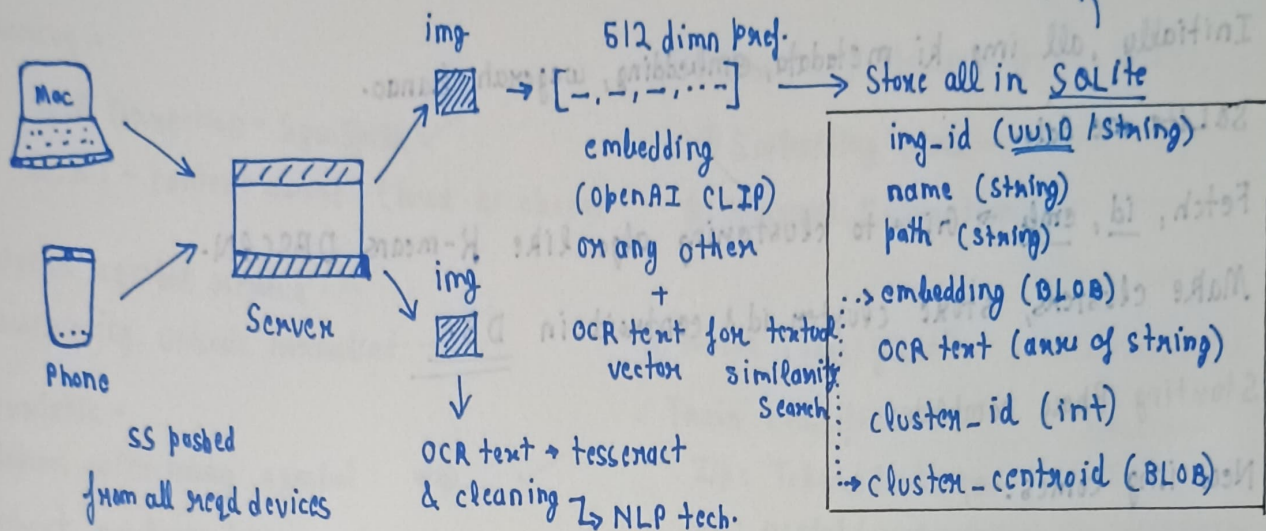


The - Watcher -

good for lakhs of samples



break text into meaningful pieces, removing useless words: is, not, hello.
[Store in SQLite, in plain string]

★ SQLite has Full-Text Search built-in.
😊 (FTSS) → Search Engine

now → [word1, word2...] X

word → rows containing it.

- Multi-word search
- AND/OR logic
- Rank, Prefix

→ Hybrid Search - Search by word, SQLite FTS return favourable result, then by FAISS, more similar results can be given. Will give supposedly good results.

• Vectorizatⁿ

• Semantic Search

• Cosine similarity instead of Euclidean dist as similar img can have diff. mnt but similar dimn. Helping in feature/semantics. [-1, 1]

Proposed system-

Initially, all img ki metadata, embedding, wagenah lianao.

Salite me dalo.

Fetch, id, emb, → Give to clustering algo like K-means, DBSCAN.

Make clusters, store cluster id & centroid in DB.

Starting Phase complete.

New img comes:

extract OCR, embedding,

• now either use cosine similarity to find k closest similar img.

• Use FAISS to do same,

→ then for the ' k ' nearest → run KNN to vote out the best fit cluster.

• Cosine-similarity - naive approach to start.

• FAISS - Fast at scale.

• New img - embedding compare against every other embedding,
 k closest ones (heap), decide cluster.

* clusters become unstable overtime, need to recluster periodically or
every 500-1000 images.

→ Tesseract - Extract text from img.

① Cleaning -

- Spell Connection - SymSpell ✓
- BERT - context aware (hood ki cheez)
- Weird symbol remove. ✓
- Lowercasing, Unicode normalize ✓
- Heuristic -
 - token with many symbol: #@_ ✓
 - Short garbage token ✓
 - repeat char: heyyy → hey ✓
 - emoji handling ★ ✓

② Tokenization

- split(): X
- spacy ✓
- NLTK ✓
- Hugging Face Transformers ✓

③ StopWord -

- Context-aware removal
- TF-IDF
- Domain-specific

④ Lemmatization:

- running → run, runs
- spacy lemmatizer

⑤ Named Entity Recognition

- No remove name, brand etc.
- spacy NER
- BERT NER

⑥ Semantic Filtering - ★

① Embedding based

② Keyword Extraction

⑦ Noise Classification -

Train classifier -

I/p: Token/Sentence

O/p: Useful/Useless

Model:

- Log Regression
- BERT

⑧ Structure Extraction -

Code block
Paragraph

⑨ Post-processing

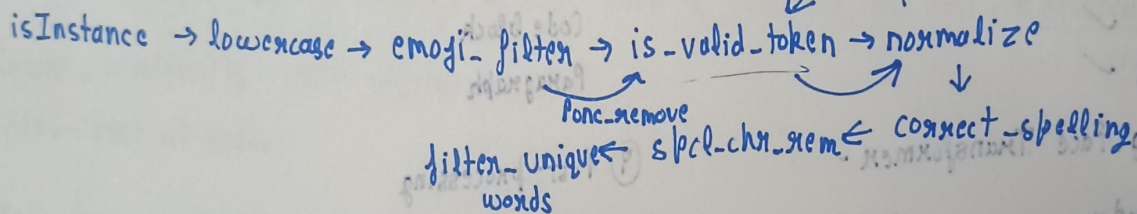
- Search Engine
- Summarizer
- Knowledge Extractor
- Classification

OCR

→ Cleaning-

- correct-spelling
- spl-chr-remove
- normalize-word (remove repetition)
- emoji-filter
- is-valid-token
- lowercase
- not is Instance (sentence, str)
- ~~lowercase~~ → ~~make~~
- (full content) • Filter-unique
- punctuatⁿ - remove

• Sequence -



- Handcode some words to not be changed or removed, as spelling correction made kb to hab, dev → de, cpu → clob } imp words changed, losing meaning

→ Beginning Cleaning-

Keeping limited to phase ① & ②.

• Ocr text → csv file

NLP Pipeline - (CampusX)

Acquisition → Text Preprocessing
↑↑

Preprocessing -

Basic -

- 1) Lowercasing
- 2) HTML Tag Removal - regex
- 3) Remove URL - regex
- 4) Remove punctuations - regex, string.punctuation

8) Hello! How are you?

Hello, !, How, are, you, ?

↑
Useless &
↑ complexity

Hello!, How, are, you?!

↑
Hello (although same but will become
diff. for our model) **Bad**

```
''' by
exclude = string.punctuation
def remove_punc(text):
    for char in exclude:
        text = text.replace(char, "")
    return text
```

slow

```
def remove_punc(txt):
```

```
    return txt.translate(str.maketrans(' ', ' ', exclude))
```

≈ 11-18 times faster

Fast

5) Chat Word Treatment - mofl, lmao, gn, lgi, asap...
or

Slang / Abbreviations

make key-value data for it in dictionary then simple check.

6) Spelling connection - Use library → symspell, NLTK, pyspellchecker, TextBlob

Hello boy, i said hello.

will form

diff tokens

7) Removing stop words - a, an, the, my, of ...

(NLTK)

8) Handling emojis -

```
def remove_emoji(txt):
```

```
    emoji_pattern = re.compile("["
```

```
        u'\U0001F600 - \U0001F64F'
```

```
        u'\U0001F300 - 1F5FF'
```

```
        u'\U0001F680 - 1F6FF'
```

```
        u'\U0001F1E0 - 1F1FF'
```

```
        u'\U0002702 - 027B0'
```

```
        u'\ 24C2 - 1F251'
```

```
    "]", flags=re.UNICODE)
```

```
    return emoji_pattern.sub(' ', text)
```

```
import emoji
```

```
emoji.demojize(txt)
```

Python is 🐍

Python is : fine:

9) Tokenization -

Tent doc → smalles, pant, like word, choi, sentence...

Bad token confuses model.

Good token very imp.

- Prefix - chr at begin

- Suffix - chr at end

- Infix - chr in b/w

- Exception - spl case rule to split string to several token or prevent token from being split when punc rules applied.

→ Word: `txt.split()`

Sentence: `txt.split(". ")`

→ `re` - patterns for splitting

→ NLTK - gives pre-built methods, ez to use.

→ Spacy - better results, ★

10) Stemming -

Inflection is modificatⁿ of word to express diff. grammatical categories, such as tense, case, voice, aspect, person, number, gender.

Stemming is process of reducing inflectⁿ in words to root forms such as mapping a group of words to same stem even if stem itself is not a valid word in lang.

→ Use NLTK.

Stemmer ↓

Porter Stemmer ✓

Snowball Stemmer

walk → walked
 walking

11) Lemmatization-

Reduces inflected words properly, ensuring root word belongs,

to lang. Root word called Lemma.

Lemma, canonical form, dictionary form, or citation form of set of words.

• WordNet dictionary - lexical dictionary

↳ needs search, hence bit slower than stemming.

Parts of speech needed like ~~verb~~ Verb.

↓
running → run

→ Assignment-

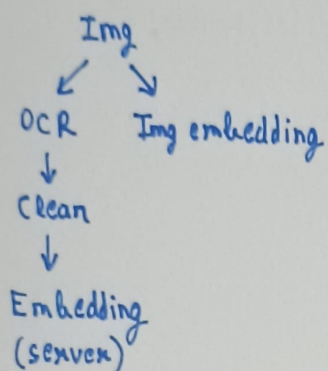
Create DS, → text preprocessing

↓

multi-class

(movie, song...)

For each img -



(Parallelized Pipeline)

→ 24/6/26

Pushed img metadata + clean_OCR to sqlite.

Successfully did test-based search based on that

→

Now using text embeddings for clustering

i) HDBSCAN

+

cosine distance using `metrics.pairwise_distances`

Param: min-sample

`min-cluster-size`

`cluster-selectn-method`

`cluster-selectn-eps`